DOCE-102 Statistical Computing
Lecture 1: Introduction - Basic Programming and Python

Nicolás Rivera

Universidad de Valparaíso
CHILE

Table of Contents

Elements of Python

Important modules: Pandas and Numpy
- Numpy
- Pandas

Control Flow

Definitions

Evaluation of Algorithms

Graphics
- Basic plots
- ggplott

At the very basic level, `Python` (also `R`) is a calculator, and operations can be directly evaluated on console.
For example

```
1+1
```

1. **Basic Arithmetic:** `+`, `-` , `*`, `/`,
2. **Powers, integer division, modulus:** `**` , `//`, `%`
3. **Relational:** >, >=, <,<=, ==
4. **Logical:** `True`, `False`, `or`, `and` , `not`

Assignation

1. In `Python` (also in `R`) everything is an object, and almost every word starting with a letter can be the name of an object
2. There are a few exception of reserved words
3. Avoid special characters, except underscores _
4. We can assign a value to an object by using =
5. For example

```
q = 1+1
Q = 3
t1 = (q==Q)
variable_2 = not t1
print(variable_2)
```

6. Use good names for your variables

Functions in Python

Functions are important if we need to execute the same code different times. **They have to return an object**.

```
def my_function(x,y,z):
        q = x+y
        print(z)
        return(q)

res = my_function(4,5,1)
print(res)
```

Be careful: functions in Python (also in R) does not specify the input, and Python will try to execute the function, independently of the arguments it receives. E.g: `my_function(True, 1, "hola")`

Lists

In `Python`, list are written between `[]`, for example

```
list_1 = [2,3]
list_2 = []
list_3 = [list_1,list_2, my_function,true, 3]
print(list_1[0]+list_1[1])
print(list_2)
print(list_3[2](1,2,3))
```

A very important function is `len` which gives us the size of the list.

Characters

We can write strings by using `''` , for example

```
name = 'Juan'
print(name)
```

In `Python` a string is essentially a list, and we can think something like

```
'Juan' = ['J','u','a','n']
```

So stuff like

```
print("juan"[3])
```

makes sense...

Objects in `Python`

Anything that can be stored in memory is an **object**. The most important thing to know about an object is its **type**

types: There are 7 very important types

- **int:** `type(5)`
- **float:** `type(4.5)`
- **bool:** `type(4==3)`, `type(False)`
- **str:** `type("hola")`
- **function:** `type(print)`, `type(my_function)`
- **lists:** `type([2,3,[2,3]])`
- **modules:** we will discuss later.. but these are what most people call "libraries" or "packages"

Table of Contents

Elements of Python

Important modules: Pandas and Numpy
- Numpy
- Pandas

Control Flow

Definitions

Evaluation of Algorithms

Graphics
- Basic plots
- ggplott

Loading Modules in Python

To use external libraries in Python, we use the `import` statement.

```
import numpy as np
import pandas as pd

# example
array = np.array([1, 2, 3])
df = pd.DataFrame({'A': [1, 2, 3]})
```

Here:

- `np` and `pd` are the names we give to numpy and pandas
- to use the functions inside the library we use "."

Introduction to NumPy

NumPy (Numerical Python) is a powerful library for numerical computing in Python. This makes Python programming very similar to `R`.

Key Features:

- Supports mathematical and logical operations on arrays.
- Optimized for performance with C-based implementation.
- Performance depends a lot on what is installed in your computer

Creating Arrays

NumPy arrays can be created from Python lists or using built-in functions.

From a list:

```
import numpy as np
arr = np.array([1, 2, 3, 4])
```

Using built-in functions:

```
zeros = np.zeros((2, 2))    # 2x2 array of zeros
ones = np.ones(5)           # Array of ones
arange = np.arange(0, 10, 2) # [0, 2, 4, 6, 8]
linspace = np.linspace(0, 1, 5) # 5 points between 0 and 1
```

Basic Operations

NumPy supports element-wise operations and linear algebra functions, very similar to R.

Element-wise Operations:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = a + b  # [5, 7, 9]
d = a * 2  # [2, 4, 6]
e = a * b  # [4, 10, 18]
```

Matrix Operations:

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
C = np.dot(A, B)  # Matrix multiplication
det_A = np.linalg.det(A)  # Determinant of A
```

Array Properties

Understanding NumPy array properties is essential for effective usage.

Shape and Size:

```
import numpy as np
A = np.array([[1, 2, 3], [4, 5, 6]])

shape = A.shape  # (2, 3) -> 2 rows, 3 columns
size = A.size    # 6 (total number of elements)
ndim = A.ndim    # 2 (number of dimensions)
dtype = A.dtype  # Data type of elements (essentially equiv
```

Reshaping and Transposing:

```
B = A.reshape(3, 2)  # Reshape to 3x2
C = A.T  # Transpose: shape becomes (3, 2)
```

Introduction to Pandas

Pandas is a powerful Python library for data manipulation and analysis. This library allow us to work with data frames in a very similar fashion to `R`.

Key Features:

- Provides `DataFrame` and `Series` objects.

Creating DataFrames

A `DataFrame` is a 2D table with labeled axes.

From a dictionary:

```
import pandas as pd
data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'Score': [90, 85, 88]}

df = pd.DataFrame(data)
```

From a CSV file:

```
df = pd.read_csv('data.csv')
```

Inspecting a DataFrame

Checking the structure of a DataFrame:

```
df.head()    # First 5 rows
df.tail(3)   # Last 3 rows
df.info()    # Summary of the DataFrame
df.describe()  # Summary statistics
```

Accessing columns and rows:

```
df['Name']   # Access a column
df.iloc[0]   # Access first row
df.loc[1, 'Age']  # Access specific value
```

Data Cleaning and Filtering

Handling missing data:

```
df.dropna()  # Remove rows with missing values
df.fillna(0)  # Replace NaN with 0
```

Filtering rows:

```
df[df['Age'] > 30]  # Rows where Age > 30
```

Converting a DataFrame to a NumPy Array

Sometimes it may be useful to treat a DataFrame as a matrix. In `R`, this is done sort of automatically, byt `Python` is slightly more structured.

```
import pandas as pd
import numpy as np

# Creating a numerical DataFrame
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}
df = pd.DataFrame(data)

# Converting to a NumPy array
array = df.to_numpy()
```

Checking the array properties:

```
print(array.shape)  # (3, 3)
print(array.dtype)  # Data type of elements
```

Table of Contents

Elements of Python

Important modules: Pandas and Numpy
- Numpy
- Pandas

Control Flow

Definitions

Evaluation of Algorithms

Graphics
- Basic plots
- ggplott

Conditional Statements: They perform code by verifying a logical statement

1. **if** (and **else**)

```
import numpy as np
x = np.random.uniform(0, 1)
if x >= 0.5:
    print("x is greater than or equals 0.5")
else:
    print("x is smaller than 0.5")
```

2. **while**

```
x = 0
trials = 0
while x < 0.5 or x > 0.6:
    x = np.random.uniform(0, 1)
    trials += 1
print(x)
print(trials)
```

Control Flow in Python

Inside a conditional statement you have to put a logical object (of length 1 ideally)

```
ready = False
trials = 0

while not ready:
    x = np.random.uniform(0, 1)
    trials += 1
    if 0.5 <= x <= 0.6:
        ready = True

print(x)
print(trials)
```

Loops: are useful to perform the same operation to different elements in a vector

```
words = ["hi", "statistics", "pencil", "computer", "Valpara
total_length = 0

for w in words:
    total_length = total_length + len(w)

print(total_length)
```

Most of the time it is more convenient to just consider indices

```
words = ["hi", "statistics", "pencil", "computer", "Valpara
total_length = 0

for i in range(len(words)):
    total_length = total_length+ len(words[i])

print(total_length)
```

Table of Contents

Elements of Python

Important modules: Pandas and Numpy
- Numpy
- Pandas

Control Flow

Definitions

Evaluation of Algorithms

Graphics
- Basic plots
- ggplott

Functions in Python

Functions are important if we need to execute the same code different times. **They have to return an object**. In Python functions are declarad with def

```
def fun(x, y, z):
    print(x)
    return [x * y + z, (x + y) / z]

res = fun(4, 5, 1)
print(res)
```

Be careful: functions in Python does not specify the input, an Python will try to execute the function, independently of the arguments it receives. Sometimes it can work, sometimes it does not. e.g. fun(np.array([4,3,2]),4,True) works

Functions in Python

There are a lot of prebuilt functions in Python, specially in the modules

1. `np.sum`, `np.mean`, `np.var`, `sd`: sum, mean, variance and standard deviation of the elements of a list-like object
2. `np.sin`, `np.cos`, `np.tan`: trigonometric functions
3. `np.sqrt` square root
4. `np.min`, `np.max`: return minimum and maximum value of a vector
5. `type`

Table of Contents

Elements of Python

Important modules: Pandas and Numpy
- Numpy
- Pandas

Control Flow

Definitions

Evaluation of Algorithms

Graphics
- Basic plots
- ggplott

Evaluation of Algorithms

- Traditionally the usual way to evaluate algorithm is to count the number of basic operations.
- Counting basic operations works well in discrete algorithm (computer science algorithms) since it is actually related with time
- In numerical/statistical algorithms this is not the case (even though a lot of CS has been applied to numerical algorithm lately)
- The most pragmatic way to evaluate an algorithm is to measure time

Evaluation of Algorithms

- In Python the module **time** can be used, in particular it has the function time
- We can observe the difference between a good algorithm and a bad one.
- **Example:** an algorithm to compute $tr(AB)$ where A is a $n \times 2n$ matrix and B is a $2n \times n$ matrix.

```python
import numpy as np
import time
n = 4000
A = np.random.normal(0, 1, (n, n * 2))
B = np.random.normal(0, 1, (2 * n, n))

# Bad Algorithm
start_time = time.time()
np.sum(np.diag(np.dot(A, B)))
end_time = time.time()
print(f"Bad Algorithm Execution Time: {end_time - start_time} seconds")

# Good Algorithm
start_time = time.time()
np.sum(A * B.T)
end_time = time.time()
print(f"Good Algorithm Execution Time: {end_time - start_time} seconds")
```

Table of Contents

Elements of Python

Important modules: Pandas and Numpy
- Numpy
- Pandas

Control Flow

Definitions

Evaluation of Algorithms

Graphics
- Basic plots
- ggplott

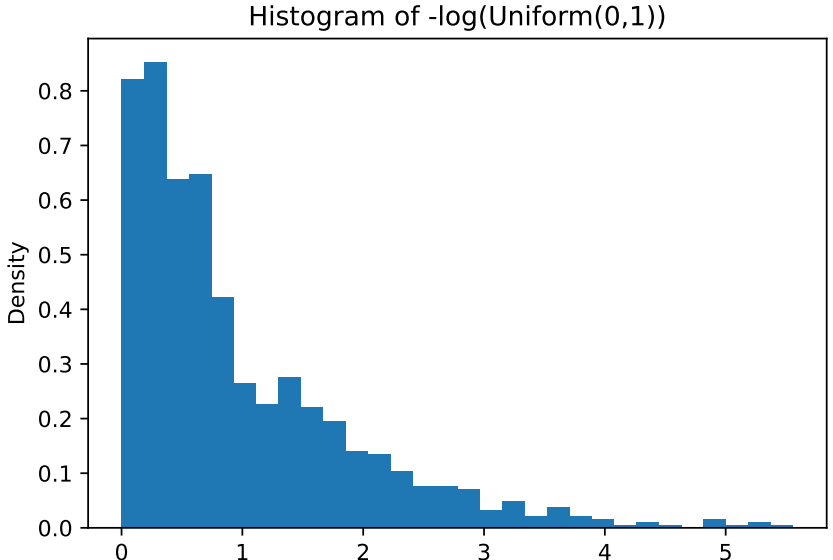
Basic Plotting: matplotlib

Matplotlib is the simplest and most used library for simple plots. For a histogram:

```
import numpy as np
import matplotlib.pyplot as plt

data = -np.log(np.random.uniform(0, 1, 1000))

plt.hist(data, bins=30, density=True)
plt.title('Histogram of -log(Uniform(0,1))')
plt.xlabel('Value')
plt.ylabel('Density')
plt.savefig('histogram_plot.pdf')
plt.show()
```



Basic Plotting functions: plot()

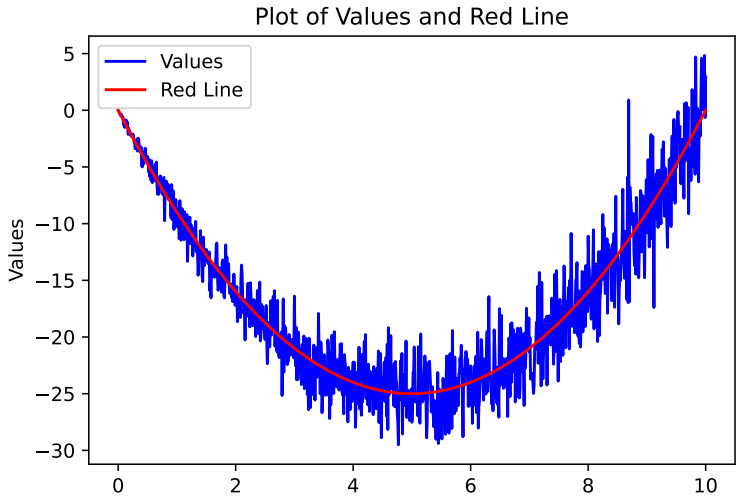
Another important function is `plot()`

```
port numpy as np
import matplotlib.pyplot as plt

time = np.arange(0, 10.01, 0.01)
values = time**2 - 10*time + np.sqrt(time) * np.random.normal(0, 1, len(time))

plt.plot(time, values, label='Values', color='blue')
plt.plot(time, time**2 - 10*time, label='Red Line', color='red')
plt.xlabel('Time')
plt.ylabel('Values')
plt.title('Plot of Values and Red Line')
plt.legend()

plt.show()
```



Ggplot

Ggplot is a system for plotting that is implemented in `R` in the package `ggplot2`, and in python in the package `plotnine`.

- ggplot is a system for declarative graphics (this means we do not have to explicitly input the graphic but a description of it).
- ggplot is based on the Grammar of Graphics by Wilkinson. There is a book with the same name which I recommend to read (at least a few chapters)
- GGplot **mostly** makes sense for statistical plots
- **In my opinion**, this is the right way of thinking about statistica plotting

ggplot

The theory behind ggplot is simple

- Data is organised in a **dataframe**
- **Rows** represent data points, and **Columns** represent features of the data
- In ggplot a lot of things are considered an axis: **x axis**, **y axis**, **colour**, **fill**, **linewidth** , **linetype**, **lineend**, **shape**, **size**, **label**, etc
- A column of the dataframe can be **mapped** to one or more axis. We cannot map two or more columns to the same axis
- A particular plot (e.g. histogram or line plot) just take the appropriate axis and draw.

ggplot, an example

Let's consider the mtcars dataframe and let's tidy it a bit.

In ggplot is very important that each column has its correct data type

```
import pandas as pd
import plotnine as p9
import statsmodels.api as sm

df = sm.datasets.get_rdataset("mtcars", "datasets").data

df["vs"] = df["vs"].astype("category")
df["am"] = df["am"].astype("category")
df["cyl"] = df["cyl"].astype(int)
df["carb"] = df["carb"].astype(int)
df = df.reset_index().rename(columns={'rownames': 'car_name'})
df.info()
```

plotnine: Step by Step

1. Create a plot canvas and indicate the dataframe to use

```
p9.ggplot(df)
```

2. Map columns of the dataframe to axes and aesthetics

```
p9.ggplot(df) +
    p9.aes(x='wt', y='mpg', color='vs', shape='am', label='names')
```

3. Plot your favorite graph: plotnine will try to make sense of it

```
p9.ggplot(df) +
    p9.aes(x='wt', y='mpg', color='vs', shape='am', label='names') +
    p9.geom_point()+
    p9.geom_text(nudge_x=0.25, nudge_y=0.25))
```

4. plot!

```
plot = (p9.ggplot(df) +
        p9.aes(x='wt', y='mpg', color='vs', shape='am', label='names') +
        p9.geom_point() +
        p9.geom_text(nudge_x=0.25, nudge_y=0.25))

plot.save("plot.pdf", width=6, height=4, dpi=300)
print(plot)
```

Melting

This is a bad dataframe. Why?

	week	type	juan	pedro	maria	daniela
1	1	mat	40	35	39	49
2	2	len	41	42	40	47
3	3	mat	41	43	39	40
4	4	len	42	42	36	20
5	5	mat	40	38	43	46
6	6	len	42	47	41	39
7	7	mat	43	41	42	38
8	8	len	41	36	36	25
9	9	mat	41	28	42	35
10	10	len	40	45	39	44

Melting

```python
import pandas as pd
df_melt = df.melt(id_vars=["week", "type"], var_name="name")
print(df_melt)
```

```
    week type    name score
1      1  mat    juan    40
2      2  len    juan    41
3      3  mat    juan    41
4      4  len    juan    42
5      5  mat    juan    40
6      6  len    juan    42
7      7  mat    juan    43
8      8  len    juan    41
9      9  mat    juan    41
10    10  len    juan    40
11     1  mat   pedro    35
12     2  len   pedro    42
13     3  mat   pedro    43
......
```

This is much better. **Why?**

7. We can also create specific `aes()` maps for a plot

```
plot = (p9.ggplot(df_melt) +
    p9.aes(x='week', y='score', color='name') +
    p9.geom_line() +
    p9.geom_point(p9.aes(shape='type'), size=4))
print(plot)
```